

Article

Optimization of Elevator Standby Scheduling Strategy in Smart Buildings

I-Ning Tsai ^{1,†}, You-Xuan Wu ^{1,†}, Yueh-Hsuan Huang ¹, Yu-Chen Chen ¹  and Jian-Jiun Ding ^{2,*} 

¹ Department of Electrical Engineering, National Taiwan University, Taipei 106319, Taiwan; b11901131@ntu.edu.tw (I.-N.T.); b11901145@ntu.edu.tw (Y.-X.W.); b11901099@ntu.edu.tw (Y.-H.H.); b11901076@ntu.edu.tw (Y.-C.C.)

² Graduate Institute of Communication Engineering, National Taiwan University, Taipei 106319, Taiwan

* Correspondence: jjding@ntu.edu.tw; Tel.: +886-2-33669652

† These authors contributed equally to this work.

Abstract

Elevator Group Control Systems (EGCSs) play a key role in managing the passenger flow and consumption of energy in modern buildings. However, existing EGCS algorithms are typically only applied to real-time passenger calls, which does not take the long-term statistics of passenger requirement into account. To address this gap, we propose a standby strategy that proactively repositioning idle elevators even if there is no passenger call. It calculates a combined score that balances the expected waiting time and the energy consumption to determine the optimal standby floors for idle elevators. We implement this strategy on a simple baseline dispatcher using the closest car algorithm and introduce tunable parameters to adjust the standby behavior. Experiments on mid-rise and high-rise building scenarios show that the standby strategy significantly reduces the average waiting time for passengers by more than 24% in both cases. Moreover, because this strategy operates independently of the core dispatcher, it can be combined with existing EGCS algorithms to further improve waiting time without compromising core energy optimizations. These findings demonstrate that proactive standby repositioning is an effective complementary approach for next-generation elevator control systems and offers a practical way to reduce waiting times under realistic office building traffic conditions.

Keywords: elevator scheduling; optimization; user experience; smart building



Academic Editor: Daswin De Silva

Received: 12 July 2025

Revised: 27 August 2025

Accepted: 10 September 2025

Published: 15 September 2025

Citation: Tsai, I.-N.; Wu, Y.-X.; Huang, Y.-H.; Chen, Y.-C.; Ding, J.-J. Optimization of Elevator Standby Scheduling Strategy in Smart Buildings. *Appl. Syst. Innov.* **2025**, *8*, 132. <https://doi.org/10.3390/asi8050132>

Copyright: © 2025 by the authors. Published by MDPI on behalf of the International Institute of Knowledge Innovation and Invention. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

In modern society, elevator systems have become an indispensable part of daily life. Whether in office buildings, residential communities, or commercial complexes, elevators play a crucial role due to the high demand for vertical transportation. The efficiency of elevator operations becomes particularly vital. During peak hours, such as the morning rush hour (up peak), lunchtime, and the evening commute hour (down peak), a large number of people need to move quickly and efficiently. The way elevators are allocated and managed directly impacts both the user experience and the overall functionality of the building. As a result, optimizing elevator allocation during these critical periods has become a key challenge.

The Elevator Group Control System (EGCS) is a central control system designed to make the optimal decision for elevator dispatch. When a passenger presses a button to call an elevator at a particular floor, this information is received by the EGCS. The system then responds to the hall call using a dispatch algorithm [1]. The EGCS algorithm takes

various factors into account, such as the direction of travel, the destination of the elevator, and the number of passengers inside. Common performance indicators for evaluating EGCS include minimizing the average waiting time (AWT), reducing the percentage of calls with long waiting times (also known as long-waiting call percentages), and improving energy efficiency [2].

The following paragraphs will introduce the development of EGCS, starting from rule-based methods, then computational intelligence approaches, and finally deep learning and reinforcement learning methods.

Traditional EGCS methods commonly use simple dispatching strategies such as the closest car algorithm (CCA) [3] and the two-handle variable (THV) duplex algorithm [4]. These approaches rely on deterministic rules and remain widely adopted in conventional systems due to their simplicity and low computational cost.

Early AI and metaheuristic techniques have also been explored in the EGCS. For example, genetic algorithms (GAs) were applied to floor-attribute-based control [5], and a GA-HCA strategy was proposed for lunchtime peak demand [6]. Fuzzy logic controllers with self-tuning achieved notable improvements in handling uncertain demand [7]. Hybrid methods, such as the combination of particle swarm optimization (PSO) and GA, further enhanced utilization and reduced waiting time [8]. These methods improve flexibility compared to purely rule-based approaches but still face the challenges of balancing multiple objectives such as waiting time, service quality, and energy consumption [1].

More recently, data-driven approaches have gained attention. Deep reinforcement learning with asynchronous actor–critic (A3C) networks achieved near-optimal dispatching under complex traffic conditions [9]. IoT-based systems such as SmartRide introduced reservation and AI-driven check-in/boarding to reduce waiting time [10]. Occupancy-aware dispatching incorporated deep learning-based capacity estimation to prevent unnecessary stops [11]. Beyond learning-based methods, optimization frameworks such as a game-theoretic coalition model [12], directional optimization for multi-floor peaks [13], and real-time matrix iterative booking with multi-sensor fusion [14] have been proposed. Furthermore, predictive scheduling with transformer networks demonstrated its effectiveness in light traffic scenarios [15]. Other reinforcement learning approaches [16,17] and sensor-based methods such as YOLO-based vision integration [18] have also been explored.

However, most of the main EGCS methods today are reactive. They only make a decision after the user presses a button [1]. These kinds of passive approaches may not always lead to the optimal performance. In contrast, the proposed approach improves this by adopting proactive standby actions even when there is no elevator call. By utilizing historical data to proactively move idle elevators to standby floors, the proposed method could reduce passenger waiting time. Furthermore, a cost function is designed to allow building operators to flexibly determine the relative importance between passenger waiting time and energy consumption according to the requirement of users. By adjusting the weight parameters in the cost function, EGCS can balance service quality and energy efficiency, achieving an optimized operational strategy. We summarize the key ideas of the proposed and other representative EGCS methods in Table 1.

Compared to the prior literature, our approach differs from (i) reactive heuristics (e.g., the CCA and the THV), which wait for calls and cannot anticipate asymmetric flows, (ii) occupancy- or camera-based [11,18,19] dispatchers, which require additional sensors and integration, and (iii) machine learning (ML)- [20–22] and reinforcement learning (RL)-based methods, which need a large dataset and much time for training and generalization. Our strategy is lightweight, training free, and modular. With a proper design of the cost function selection, it can reduce the future chase distance relative to purely reactive dispatch, and it does not need to consume more energy in localized-burst scenarios.

Table 1. Representative EGCS methods.

Method	Key Idea
THV duplex algorithm [4]	Using two-handle variables to balance elevator assignments with simple rule-based logic.
Fuzzy logic control [7]	Applying fuzzy rules with self-tuning to handle uncertain passenger demand.
Closest Car Algorithm (CCA) [3]	Assigning the nearest idle elevator to the passenger request.
Genetic algorithms [23]	Genetic algorithms and a fitness function.
Optimization Models [12–14]	Game theory, directional optimization, and real-time matrix iterative booking.
Deep learning (DL) [11,15,20–22]	Deep learning, long-short-term memory (LSTM), transfer learning, and the transformer.
Reinforcement learning (RL) [9,16,17]	Learning a unified dispatching policy from simulated traffic data.
Sensor-based dispatching [11,18,19]	Integrating sensor or vision data to improve allocation decisions.
Proposed standby strategy (ours)	Proactively repositioning idle elevators using a cost function balancing waiting time and energy consumption.

The remainder of this article is organized as follows. Section 2 describes the materials and methods, including the simulation environment, the detail of the dispatcher design, and the formulation of the proposed standby strategy. Section 3 presents the experimental results, including baseline comparisons, ablation studies, and a discussion of the key findings. Finally, Section 4 concludes the paper and outlines the future work.

2. Materials and Methods

An advanced elevator standby and dispatch strategy that is suitable for both high-rise and mid-rise building scenarios is proposed. This section describes the design of the baseline dispatcher, the formulation of the standby strategy, the adopted evaluation metrics, and the simulation environment. By isolating the standby mechanism from other scheduling components, our setup ensures a fair and controlled comparison across different configurations and traffic patterns.

2.1. Simulation Environment and Settings

To evaluate the proposed standby strategy under realistic conditions, we constructed a custom simulation using Python 3.10 and the SimPy4.1.1 discrete-event simulation library. Two building configurations were considered: a mid-rise office building with 10 floors and 2 elevators, and a high-rise office tower with 20 floors and 4 elevators. In both cases, each elevator travels at a speed of 12 floors per minute and accommodates up to 8 passengers.

The simulation spans five consecutive weekdays (Monday to Friday), totaling 7200 min. From Figure 1, which is the daily traffic profile adapted from the work of Hautamäki et al. [24], passenger arrivals are modeled using a superposition of Gaussian functions that approximate daily office traffic patterns. This profile captures the three canonical peaks—morning up-peak, lunchtime inter-floor traffic, and evening down-peak—while stochastic variation and directional biases are added to avoid unrealistic symmetry. Although weekends are not explicitly simulated, their demand patterns largely resemble off-peak or late-night periods, where the standby strategy still operates effectively. We adopted Gaussian functions because they are easier to analyze and can be augmented with stochastic variation and directional bias. Although a Gaussian function is symmetric, its superposition is able to generate some asymmetric and irregular flows to some extent:

$$\lambda(t) = A_1 \cdot \exp\left(-\frac{(t - \mu_1)^2}{2\sigma_1^2}\right) + A_2 \cdot \exp\left(-\frac{(t - \mu_2)^2}{2\sigma_2^2}\right) + A_3 \cdot \exp\left(-\frac{(t - \mu_3)^2}{2\sigma_3^2}\right) + \epsilon. \quad (1)$$

Here, t is the time in minutes (modulo 1440), μ_i and σ_i denote the mean and standard deviation of each peak (e.g., $\mu_1 = 480$ for 08:00), A_i is the amplitude, and ϵ is a baseline rate that ensures minimal off-peak activity.

Passenger trips are categorized into three types:

- Incoming traffic: Passengers travel from the ground floor to upper floors.
- Outgoing traffic: Passengers descend from upper floors to the ground floor.
- Inter-floor traffic: Passengers move between various upper floors and the directions can be upward or downward.

These flows align with classical elevator traffic patterns:

- Morning up-peak (7:00–10:00): predominantly upward movement.
- Lunchtime peak (11:00–14:30): mainly inter-floor trips.
- Evening down-peak (16:00–21:00): predominantly downward travel.

This time-varying demand collectively provides a realistic basis for evaluating elevator performance in typical office buildings.

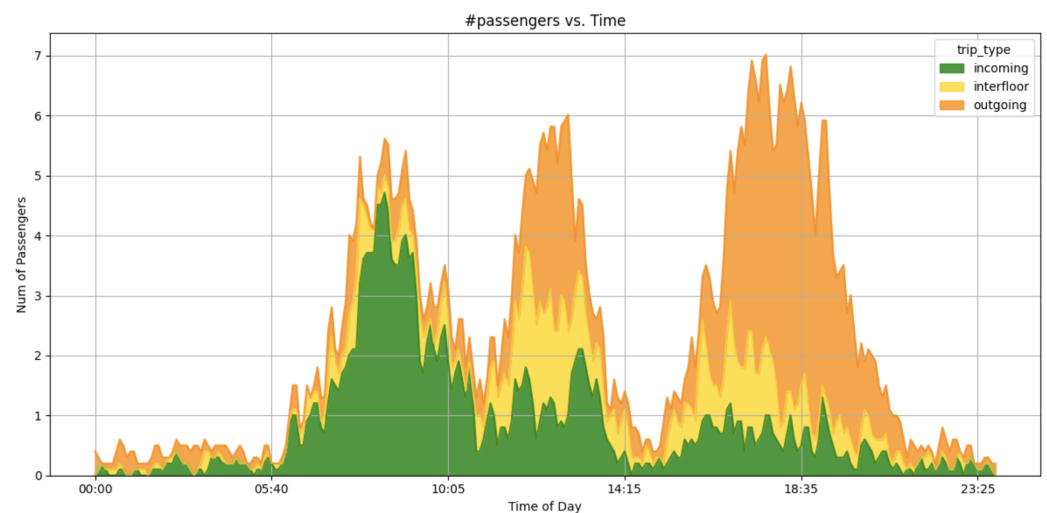


Figure 1. Daily traffic profile adapted from Hautamäki et al. [24].

2.2. Dispatcher Design

To provide a controlled and interpretable baseline for evaluating the effectiveness of the proposed standby strategy, we implemented a simple yet logically structured rule-based dispatcher. The dispatcher mimics the conventional elevator assignment behavior and follows a deterministic decision flow.

1. Idle Elevators Preference: Among all idle elevators, the one closest to the passenger's origin floor is selected.
2. Fallback to Busy Elevators: If no idle elevator is available, the system selects the elevator currently in service with the fewest queued requests. In case of a tie, the one with the shortest distance to the request floor is chosen.

This dispatcher serves as a consistent foundation for both baseline evaluation and comparative experiments. Crucially, we add the standby strategy on top of the same dispatcher without altering its core logic. This separation allows us to attribute performance improvements solely to the standby mechanism.

Conceptually, the dispatcher handles the behavior of the elevator after a passenger presses a button (that is, reacting to demand), while the standby strategy acts before whenever a call is made (that is, anticipating demand). Since these two modules are logically independent, the proposed standby mechanism can be seamlessly integrated into

existing Elevator Group Control Systems (EGCSs) with minimal computational overhead or structural changes.

A detailed description of the dispatch logic is provided in Appendix A.

2.3. Standby Strategy

Most traditional elevator systems respond only after the user presses a button. In contrast, the proposed standby strategy introduces a lightweight proactive module that repositions idle elevators to likely demand hotspots before a call occurs. The complete decision pipeline is illustrated in Figure 2.

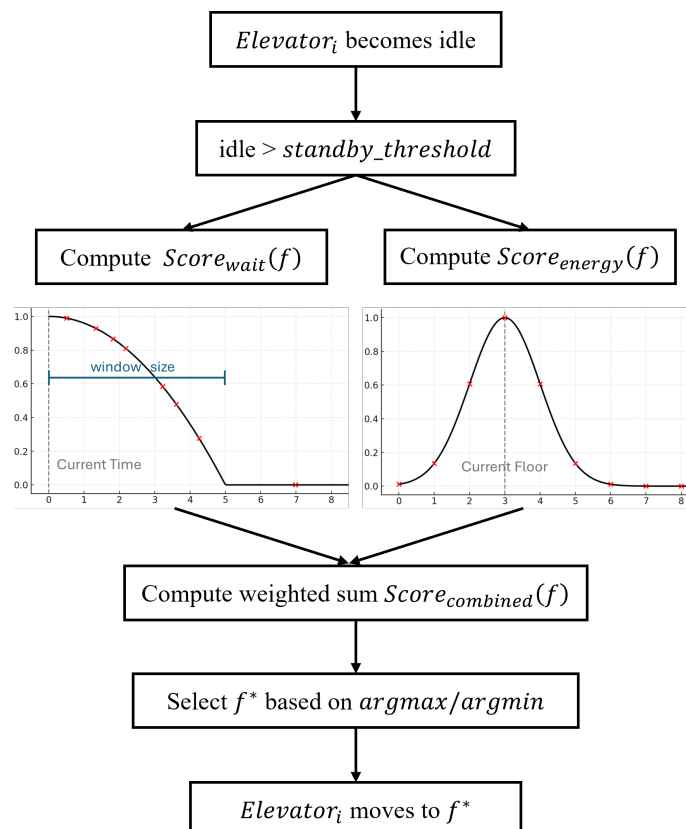


Figure 2. Overview of the standby strategy workflow. When an elevator becomes idle and exceeds the predefined standby threshold, it triggers score computation across all floors. $Score_{wait}$ reflects the recent call activity using a decay function, while $Score_{energy}$ penalizes long movement based on Gaussian proximity. The two scores are then combined into a weighted score per floor. The elevator is repositioned to the optimal floor f^* selected via either the greedy or the optimal cost strategy.

Unlike traditional elevator dispatchers that respond only after a call is made, our strategy proactively positions an idle elevator on a floor where future demand is likely to occur. When an elevator remains idle beyond a threshold (default: 0.2 min), it calculates an optimal standby floor based on recent traffic history.

The standby mechanism is designed as a lightweight and independent module that can be added on top of any existing EGCS without altering its core logic. By handling the “pre-call” phase of the elevator behavior, it complements the traditional dispatcher, which operates in the “post-call” phase. The standby decision process consists of two components: (i) a floor scoring mechanism and (ii) optional enhancement techniques for dynamic adaptation.

2.3.1. Standby Floor Scoring

The core of the proposed standby mechanism lies in selecting an appropriate floor f^* to reposition when an elevator becomes idle. This is achieved by computing a weighted score that balances two competing objectives: (i) minimizing passenger waiting time and (ii) reducing unnecessary energy consumption. The combined score for each candidate floor f is defined as

$$\text{Score}_{\text{combined}}(f) = w \cdot \text{Score}_{\text{wait}}(f) + (1 - w) \cdot \text{Score}_{\text{energy}}(f), \quad (2)$$

where $w \in [0, 1]$ is a tunable parameter that adjusts the relative importance of the recent demand versus the energy cost.

The parameter w can be applied to prioritize different performance goals. A higher w places more emphasis on recent passenger activity, effectively prioritizing user experience to reduce expected waiting time. In contrast, a lower w gives more weight to energy saving, which is related to reducing the moving distance of the elevator. This tunable parameter offers building managers flexibility. It can balance the trade-off between service quality and energy consumption.

Demand-Based Term: $\text{Score}_{\text{wait}}(f)$ reflects the recent arrival activity at floor f , using a decay-weighted sum over historical passenger requests:

$$\text{Score}_{\text{wait}}(f) = \sum_{t \in H(f)} \phi(t), \quad (3)$$

where $H(f)$ denotes the set of recent arrival timestamps at floor f and $\phi(t)$ is a function to reduce the effect of older requests. Three decay modes are supported:

- Linear: $\phi(t) = \max(0, 1 - \frac{t}{\lambda W})$,
- Exponential: $\phi(t) = e^{-\frac{\alpha t}{\lambda W}}$,
- Quadratic: $\phi(t) = \max(0, 1 - (\frac{t}{\lambda W})^2)$,

where t is the elapsed time since request, W is the window size, and λ is a scaling factor controlling the decay rate.

Each decay mode reflects a different assumption about how strongly recent arrivals should influence the standby decision. Linear and quadratic functions impose hard cutoffs, while the exponential form provides a smooth and long-tailed decay. The quadratic mode emphasizes very recent requests due to its faster decay rate, making it suitable for fast-changing demand. In contrast, the exponential mode retains more memory of past activity and is useful in more stable traffic patterns. The flexibility in decay design allows the proposed strategy to be adaptive to various traffic dynamics.

Energy-Based Term: $\text{Score}_{\text{energy}}(f)$ penalizes repositioning to the distant floor via a normalized Gaussian function:

$$\text{Score}_{\text{energy}}(f) = \exp\left(-\frac{(f - f_0)^2}{2\sigma^2}\right), \quad (4)$$

where f_0 is the current floor of the idle elevator, and σ is the standard deviation controlling the sensitivity to distance. This formulation favors the nearby standing floor to reduce the moving distance.

Final Selection: Once the combined score $\text{Score}_{\text{combined}}(f)$ has been calculated for each candidate floor f , the standby target floor f^* is selected using one of two decision strategies:

$$f^* = \arg \max_{f \in F} \text{Score}_{\text{combined}}(f) \quad (\text{Greedy selection}), \quad (5)$$

The greedy strategy simply selects the floor with the highest score among all candidates. It does not consider the scores of the nearby floors. Although this method is simple and fast, it can be short-sighted, especially when there is a cluster of high scores around a certain region.

2.3.2. Enhancement Techniques

To further improve the flexibility and robustness of the proposed standby floor selection strategy, we introduce several optional enhancement modules that allow the strategy to adapt to different building and traffic characteristics.

Optimal Cost Function Strategy: In addition to the greedy selection scheme that chooses the floor with the highest score, as in (5), we offer an alternative cost-minimizing scheme based on the weighted sum of the original scores of all other floors, where their distances serve as weights. This method selects the standby floor f^* that minimizes the total weighted travel cost to all other floors:

$$f^* = \arg \min_{f \in F} \sum_{x \in F} |x - f| \cdot \text{Score}_{\text{combined}}(x) \quad (6)$$

This objective function effectively estimates the center of gravity of floor demand: the score of each floor represents a discrete mass, and the optimal standby floor minimizes the total “moment” relative to all others. Compared to greedy selection, this approach avoids extreme placements and provides better overall coverage of active zones, as the example in Figure 3.

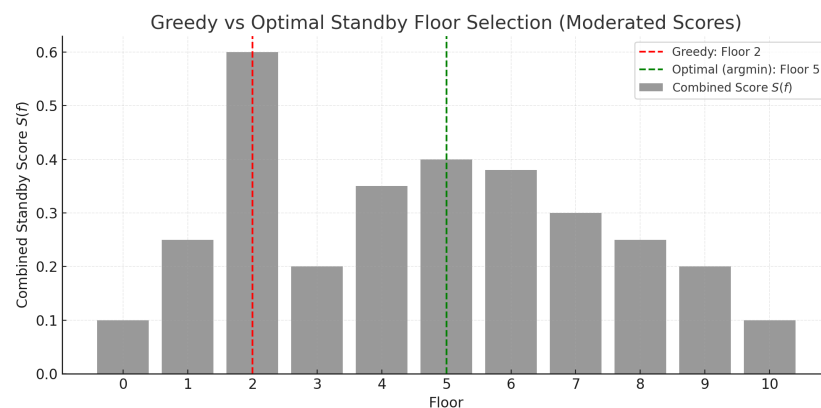


Figure 3. Illustration of standby floor selection using the combined score function. The greedy strategy selects the floor with the highest score as the red line, while the optimal cost function strategy selects the floor that minimizes the total expected travel cost as the green line. Although the 2nd floor has the highest score, the 5th floor better balances a wider spread of demand and is the optimal solution for elevator repositioning.

Dynamic Window Size: The time window W used to calculate $\text{Score}_{\text{wait}}(f)$ can be dynamically adjusted based on the load of the system. When the request activity is frequent, W is shortened to increase responsiveness; when the loading is light, W becomes longer to stabilize decisions. A simple inverse-proportional rule is used:

$$W = \max\left(W_{\min}, \frac{W_{\text{base}}}{1 + \rho}\right), \quad (7)$$

where ρ is the number of recent requests, and $W_{\min}$ is a lower bound to avoid over-shrinking.

Type Parameter λ : In (3), the sharpness of the decay function in $\text{Score}_{\text{wait}}(f)$ is controlled by the parameter λ . A smaller λ results in a steeper decay, making the system more

sensitive to very recent arrivals. This parameter can be applied to control the effects of recent events and long-term statistics. It provides additional flexibility in shaping the standby behavior. It can be tuned according to the traffic characteristics of a specific building.

2.4. Evaluation Metrics

To comprehensively assess the impact of the standby strategy, we apply both passenger-centric and system-centric metrics, along with standby-specific indicators. These metrics are computed based on simulation logs and aggregated across multiple runs for robustness.

Passenger-Centric Metrics:

- Average Waiting Time (AWT): It means the duration between the arrival time of the passenger and the pickup time of the elevator.
- Wait Time Percentiles (Q1, Q2, Q3): They mean the first, median, and third quartiles of the waiting time distribution.
- Maximum and Minimum Wait Times: They correspond to the waiting time in the extreme cases.

Standby-Specific Metrics:

- Standby Interruptions: This means the number of standby movements that were interrupted due to new requests.
- Standby Success Rate: It is the fraction of standby movement that reduced the distance to the next pickup compared to the remaining idle.
- Standby Score: It is the average relative improvement in distance due to standby repositioning, normalized by total standby movement.

System-Centric Metrics:

- Energy Consumption: While the total floors traveled by all elevators can be adopted as a proxy for energy consumption, we further derive a more detailed formulation that explicitly accounts for the elevator car mass, passenger load, and start/stop operations (detailed derivation in Appendix B):

$$E_{\text{total}} = E_{\text{car,up}} + E_{\text{passenger,up}} + E_{\text{start/stop}} \quad (8)$$

These metrics allow for a holistic comparison between the baseline and standby enabled systems, capturing both operational efficiency and service quality.

3. Results

This section presents the experimental setup, the performance of baseline and standby strategies, a detailed ablation study, and a discussion of key findings.

3.1. Performance with the Standby Strategy Versus the Baseline

3.1.1. Parameter Settings

The key parameters used in the experiments are as follows:

- standby_threshold = 0.2
- w = 0.7
- std = 1.0
- window_size = 240.0
- decay_mode = "quadratic"
- optimal_cost_function = True

These parameters correspond to the standby strategy described in Section 2. These parameters were selected based on ablation studies and preliminary grid search to jointly optimize the passenger waiting time and the elevator travel distance.

3.1.2. Performance Comparison

Each result is averaged over three runs using the datasets generated with the same distribution. We evaluated the proposed standby strategy against three widely adopted dispatching baselines: the THV duplex algorithm [4], the fuzzy logic-based method [7], and the closest car algorithm [3]. We also include a naive *center-based standby* method that always repositions idle elevators to the middle floor.

Figure 4 presents the average waiting time (AWT) across two building configurations, and Table 2 provides detailed statistical results.

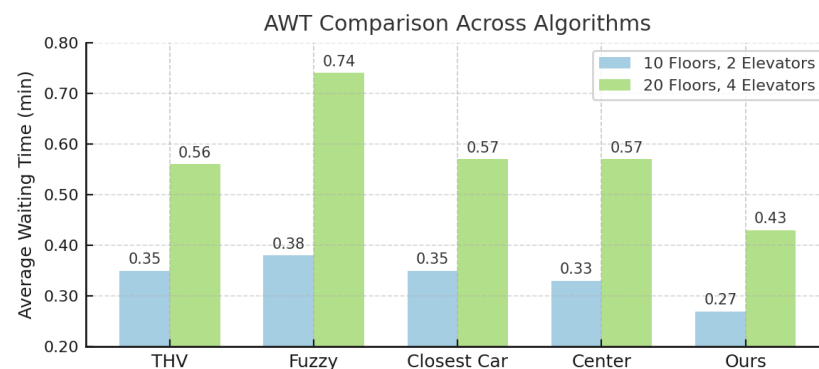


Figure 4. Comparison of the average waiting times of the proposed method and baseline algorithms under two scenarios: a 10-floor building with 2 elevators and a 20-floor building with 4 elevators.

Table 2. Detailed statistics of waiting time and energy consumption across algorithms.

Scenarios	Algorithm	AWT $\pm$ 95% CI	Q1	Q3	Max	Energy
10 Floors, 2 Elevators	THV	0.350 ± 0.014	0.15	0.52	1.33	95.11
	Fuzzy	0.380 ± 0.016	0.15	0.56	1.55	89.13
	Closest Car	0.350 ± 0.015	0.15	0.53	1.43	96.94
	Center	0.330 ± 0.011	0.18	0.44	1.33	128.27
	Proposed	0.270 ± 0.012	0.11	0.38	1.36	119.86
20 Floors, 4 Elevators	THV	0.560 ± 0.027	0.17	0.86	2.77	185.01
	Fuzz	0.740 ± 0.038	0.19	1.20	3.84	164.81
	Closest Car	0.570 ± 0.027	0.17	0.88	2.82	187.06
	Center	0.570 ± 0.018	0.31	0.84	2.57	262.83
	Proposed	0.430 ± 0.021	0.14	0.64	2.66	243.66

Scenario 1: 10 Floors, 2 Elevators

Among all evaluated methods, the proposed strategy achieves the lowest AWT (0.270 min), representing a 22.9% reduction compared to the closest car algorithm. Quartile statistics (Q1, Q3) and maximum waiting time are also improved, indicating not only lower mean latency but also reduced variability. This shows the robustness of the method in user experiences.

Scenario 2: 20 Floors, 4 Elevators

In the larger configuration, our method continues to outperform all baselines, with an AWT of 0.430 min—24.6% lower than the closest car algorithm. The third quartile drops from 0.88 to 0.64 min, demonstrating improved tail latency.

As expected, the proactive repositioning incurs higher energy consumption due to additional movements. However, when combined with energy-efficient reactive dispatchers, it can still achieve AWT while keeping the energy overhead moderate by flexibly tuning the scoring parameter w .

Overall, the proposed strategy offers a practical and effective enhancement to existing dispatchers, striking a favorable balance between service quality and energy efficiency.

3.1.3. Illustration of Elevator Trajectories

Figure 5 illustrates the distribution of standby floors selected by elevators during three distinct time periods: morning up-peak (08:00–09:00), lunchtime (14:00–15:00), and evening down-peak (19:00–20:00). Each bar represents the number of times that elevators are repositioned on a specific floor within the time period. The result reveals a clear temporal pattern: during the morning, standby floors concentrate on the lower levels (floors 0–2) to anticipate upward commuter flow; at noon, the distribution is more dispersed and moderate; and in the evening, the standby strategy favors mid-to-high floors (floors 3–7), aligning with the downward demand of office workers to leave the building. This statistical result supports the proposed idea to design an adaptive elevator repositioning scheme according to dynamic demand over time.

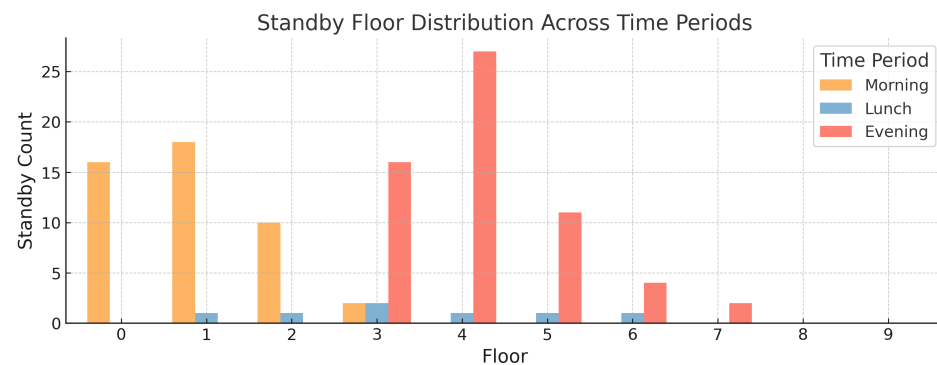


Figure 5. Standby floor frequency by time period: morning (08:00–09:00), lunch (14:00–15:00), and evening (19:00–20:00).

Figure 6a,b show the trajectories of each elevator between 08:00 and 08:20 for the closest car algorithm dispatcher and the closest car algorithm with the proposed standby dispatcher (standby dispatcher for short), respectively, using the same traffic dataset. The trajectories indicate periods of servicing and response for the closest car algorithm dispatcher and servicing, response, standby, and standby (interrupted) for the standby dispatcher. Since the traffic in this period has a typical incoming traffic style, most passengers travel from the ground floor to upper floors. The standby dispatcher dynamically repositions idle elevators to the ground floor most of the time to reduce waiting times.

Figure 7a,b illustrate the trajectories for each elevator during the evening peak, from 19:00 to 19:20. In this outgoing traffic period, most passengers travel from upper floors back to the ground floor. The standby dispatcher dynamically selects standby floors that minimize the cost function based on recent traffic patterns. It leads to intermediate floors such as the fourth or fifth floor being often chosen.

To further illustrate the impact of standby repositioning, we annotate one representative request instance using shaded regions in Figures 6 and 7, where green/gray circles are passenger requests, black circles are elevator stops/pickups, gray lines are baseline waiting events, green lines are standby strategy waiting events, blue triangles are standby initial positions, red crosses are interrupted standby, gray bars denote the original waiting time without standby, blue bars mark the proactive standby movement of the elevator

prior to the request, and green bars indicate the reduced waiting time after standby is introduced. Each selected instance corresponds to the same user call in both the baseline and the standby scenario. As observed, the green bar is significantly narrower than the gray bar, visually confirming that the standby dispatcher effectively shortens the user waiting time by anticipating the demand and pre-positioning idle elevators.

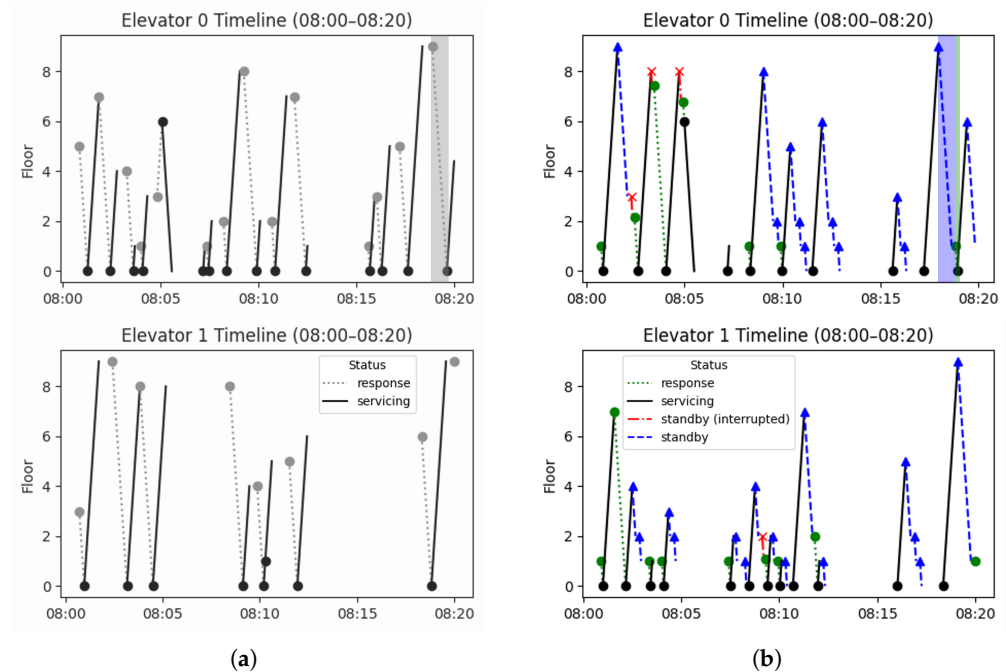


Figure 6. Elevator trajectories (08:00–08:20) of a 10-floor building with two elevators using (a) the closest car algorithm dispatcher, and (b) the proposed standby dispatcher. The standby strategy favors lower floors during up-peak.

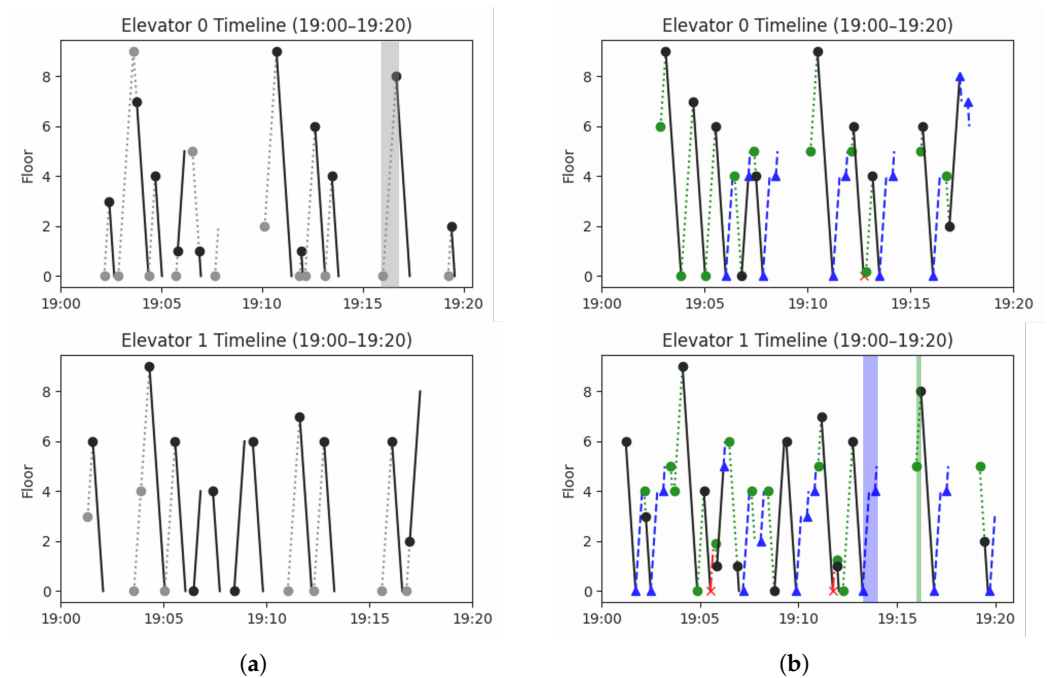


Figure 7. Elevator trajectories (19:00–19:20) of a 10-floor building with two elevators using (a) the closest car algorithm dispatcher, and (b) the proposed standby dispatcher. It tends to position the elevator to upper floors during down-peak.

3.2. Ablation Study of Internal Mechanisms

3.2.1. Effect of Standby Triggering Threshold

As described in Section 2.3, the `standby_threshold` defines the period of idleness before an elevator is repositioned. Increasing this threshold results in longer waiting times for passengers but fewer floors traveled as in Figure 8. When it becomes very large, the system approaches the non-standby case. From Figure 8, one can see that, as the standby threshold increases, the energy consumption decreases but the AWT increases.

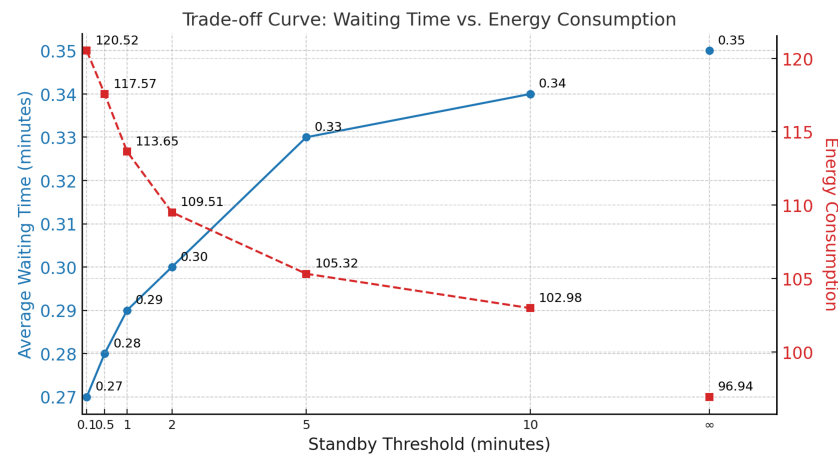


Figure 8. Trade-off between average waiting time and energy as the standby threshold changes. The final point (∞) represents the baseline scenario without standby repositioning.

3.2.2. Effect of the Optimal Cost Function

As described in Section 2.3.2, we minimize the cost function to determine the optimal floor for elevator repositioning. As shown in Table 3, after optimization, the mean waiting time is reduced, and a narrower waiting time distribution is produced, with fewer extreme values. It also reduces the total number of floors traveled and improves energy efficiency.

Table 3. Key performance metrics with and without the optimal cost function.

Metric	Without Cost Function	With Cost Function	Change
Mean AWT (min)	0.33	0.27	−18.2%
IQR (Q3–Q1, min)	0.43	0.27	−37.2%
Energy	115.41	119.86	+3.9%

3.2.3. Effect of Weight w to Balance the Waiting Time and the Energy Term

As described in Section 2.3.1, the weight for $Score_{waiting}$ is w and for $Score_{energy}$ is $1 - w$. As w increases from 0.2 to 0.8, the waiting time decreases while energy increases, reflecting the trade-off between waiting time and energy savings. The parameter w can be customized by users according to their preference for lower waiting time or higher energy efficiency. In Figure 9, we show the trade-off between the average waiting time and energy consumption quantitatively. As the parameter w increases, the energy consumption increases but the AWT decreases.

3.2.4. Effect of Decay Mode and Window Size

The parameters `decay_mode` and `window_size` can be applied to shape the sliding window for calculating $score_{standby}$. Experiments show that varying `decay_mode` and `window_size` does not significantly affect waiting time. However, as shown in Figure 10a, a larger `window_size` consistently reduces the energy. Likewise, as shown in Figure 10b,

changing decay_mode from the exponential mode to the linear to quadratic mode also leads to a decrease in energy.

This trend results from how the weighting emphasizes recent data points. A larger window_size slows the decay of weights, making the event fed for a long time before having more influence, which helps to smooth out extreme values and prevent unnecessary repositioning. The parameter decay_mode has similar effect. As the decay becomes flatter, the old data has more influence, and the system is more robust to the effect of extreme cases.

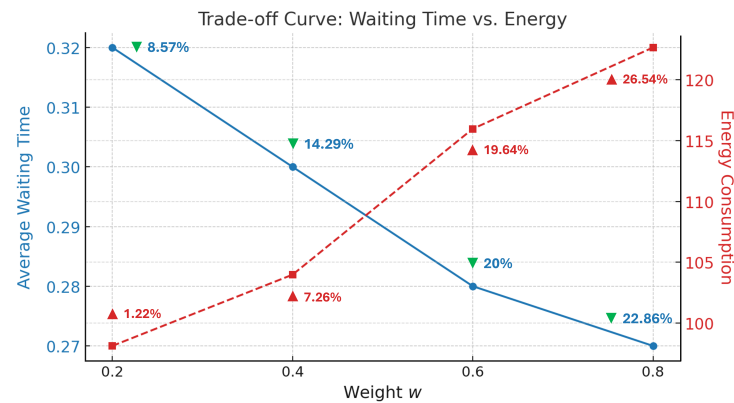


Figure 9. Trade-off between the average waiting time and energy as w changes. Percentage annotations indicate relative improvement or deterioration from the baseline (i.e., $w = 0$) without standby strategy.

However, if window_size is too large, the waiting time increases. This is expected, as an excessively large window may capture outdated or less relevant information, leading to poor standby decision and worsened system performance.

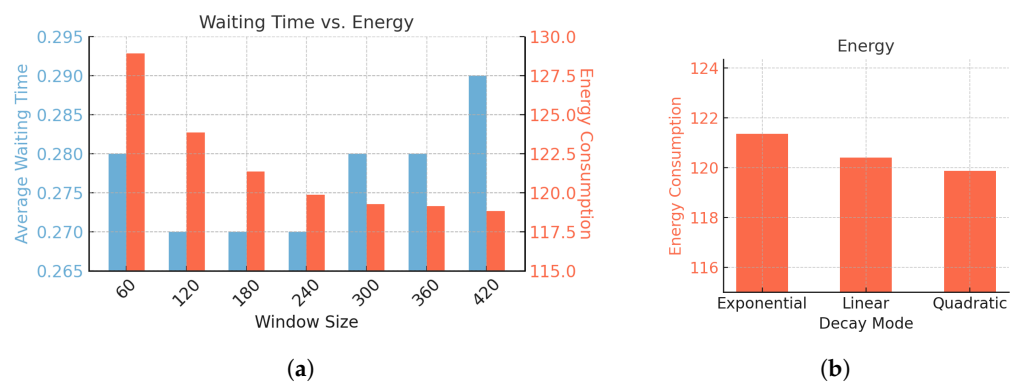


Figure 10. (a) Effect of the window size on average waiting time and energy consumption. (b) Energy consumption under different decay modes.

3.3. Discussion

After analyzing key metrics—including average standby moving floors, total energy consumption, number of interruptions, mean waiting time, and energy—we observe correlations among them. When the average standby moving floors increase, the total standby moving floor and thus the energy also rise, leading to higher overall energy consumption. Therefore, it is important to avoid overly large average standby moving floors. However, an extremely low average implies that the standby strategy is rarely triggered, which can also lead to higher waiting times.

This demonstrates the complex relationship between standby movement, interruptions, energy consumption, and passenger waiting time. Experimental results show that while the standby strategy can significantly reduce waiting time, excessive standby move-

ment can also lead to higher energy use and more interruptions. This highlights the importance of parameter selection to make a practical balance between waiting time reduction and energy consumption.

Based on the above observations, we suggest a methodology to select hyperparameters. In practice, the weight parameter w can be chosen according to the primary objective of the operator: higher values are preferable in user-centric environments, such as commercial offices, where minimizing waiting time is critical, while lower values are more suitable in energy-sensitive buildings. Additionally, adopting smaller values of the `standby_threshold` and the window size can further decrease waiting time and is suitable for the scenario where the demand should be addressed faster.

4. Conclusions

A novel standby strategy for Elevator Group Control Systems (EGCS) was proposed to address a problem that has received little attention in existing work: proactively positioning idle elevators in anticipation of future requests. Unlike conventional dispatching systems, which respond only after requests are made, the proposed approach calculates a dynamic *standby score* that combines both waiting time and energy consumption to determine the optimal standby floor whenever an elevator is idle.

Moreover, the ways to design the cost function, the decay mode, and the standby triggering mechanism were also proposed to ensure practical effectiveness. They are helpful for balancing the trade-off between repositioning behavior and system stability.

Experimental results demonstrate that the proposed standby strategy can significantly reduce the waiting time of passengers in both small and large building scenarios. The proposed strategy is flexible and can be integrated into any existing EGCS framework.

For future work, several extensions are envisioned. These include validating the approach in large-scale real-world buildings and adapting the strategy to more complex traffic patterns such as those in airports or multi-use facilities.

Note that the adopted Gaussian model with stochastic variation already produces some asymmetric and irregular flows. Therefore, the proposed approach is applicable to asymmetric flow and midday peak scenarios. The limitation of this work is that the evaluation was performed under centralized traffic scenarios. It is essential to further improve the proposed approach and take more complex and heterogeneous traffic patterns into consideration. In fact, since the proposed standby module is lightweight and modular, it can be integrated with more advanced dispatchers and extended to larger-scale systems without modifying their core logic.

In summary, this work highlights the practical value of proactive standby repositioning. It can be a powerful technique for next-generation elevator control systems.

Author Contributions: Conceptualization, I.-N.T., Y.-X.W., and J.-J.D.; Methodology, I.-N.T., Y.-X.W., Y.-H.H., and Y.-C.C.; Software, Y.-X.W. and I.-N.T.; Validation, Y.-X.W. and I.-N.T.; Formal Analysis, Y.-X.W. and I.-N.T.; Investigation, Y.-X.W. and I.-N.T.; Data Curation, Y.-X.W. and I.-N.T.; Writing—Original Draft Preparation, Y.-X.W., I.-N.T., and Y.-H.H.; Writing—Review and Editing, J.-J.D., I.-N.T. and Y.-X.W.; Visualization, I.-N.T. and Y.-X.W.; supervision, J.-J.D.; project administration, J.-J.D.; funding acquisition, J.-J.D. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the National Science and Technology Council, Taiwan, under the contract of NSTC 113-2221-E-002-146.

Data Availability Statement: The simulation data supporting this study were generated using a custom Python simulator based on a daily traffic profile adapted from Hautamäki et al. The passenger generator and the full simulation code will be found at <https://github.com/clarawu0608/>

[Optimization-of-Elevator-Standby-Scheduling-Strategy.git](#), accessed on 8 June 2025. No external dataset was used beyond this synthetic traffic model.

Acknowledgments: The authors thank the National Science and Technology Council, Taiwan, for their support.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Appendix A. Dispatcher Design

Appendix A.1. Closest Car Algorithm Dispatcher (Baseline Dispatcher)

The closest car algorithm dispatcher uses a straightforward and reasonable assignment logic widely adopted in conventional elevator systems. Its core principle is as follows.

1. Prioritize idle elevators:
 - For any incoming request, find all idle elevators.
 - Select the one closest to the original floor of the passenger.
2. Fallback to servicing elevators if none are idle:
 - For all elevators in service, estimate:
 - the number of queued requests;
 - the distance from the last scheduled stop to the original floor of the new passenger.
 - Pick the elevator according to the number of requests and the distance.

This design ensures that idle elevators are always preferred first. It minimizes the waiting time when there is available capacity. Note that the proposed dispatcher always forms a candidate set of available elevators. If an elevator is blocked or unavailable, the elevator is excluded and the request is reassigned to the remaining elevators following the closest-car fallback logic.

Appendix A.2. Standby Dispatcher

To test the proposed standby strategy in isolation, we extend the baseline logic by introducing the standby behavior but keep all other mechanisms unchanged. Additional rules for standby dispatch are as follows.

1. Include standby elevators in the idle pool:
 - Treat elevators moving to standby positions the same as idle ones with the following additional checks:
 - Select the elevator closest to the original floor of the passenger.
 - If multiple elevators have the same distance, the one that is moving toward the passenger is preferred. The second choice is the idle one. The elevator that is moving away from the passenger is less preferred.
2. Fallback scenario:
 - If no idle or standby elevator is available, then the original closest car algorithm is applied.
3. Interrupt standby if needed:
 - If an elevator is moving to the standby floor but is selected for a new request, then the movement is interrupted immediately and the elevator is switched to the idle mode.

Appendix B. Energy Consumption Formulation

Let:

- m_c : car mass (kg)
- m_p : average passenger mass (kg)
- g : gravitational acceleration (m/s^2)
- h : floor height (m)
- $E_{\text{start}}, E_{\text{stop}}$: constant start/stop energies (J)
- N_{seg} : total number of movement segments
- F_{car} : total floors traveled by each car
- $F_{\text{pass}}^{(i)}$: floors traveled by passenger i

Car traveling energy

$$E_{\text{car}} = m_c g (F_{\text{car}} \cdot h)$$

Passenger traveling energy

$$E_{\text{passenger}}^{(i)} = m_p g (F_{\text{pass}}^{(i)} \cdot h)$$

Summing across all upward passenger trips yields $E_{\text{passenger}}$.

Start/stop energy

$$E_{\text{start/stop}} = N_{\text{seg}} \cdot (E_{\text{start}} + E_{\text{stop}})$$

Total Energy

$$E_{\text{total}} = E_{\text{car,up}} + E_{\text{passenger,up}} + E_{\text{start/stop}}$$

References

1. Hanif, M.; Mohammad, N. Metaheuristic algorithms for elevator group control system: A holistic review. *Soft Comput.* **2023**, *27*, 15905–15936. [\[CrossRef\]](#)
2. Fernandez, J.R.; Cortes, P. A survey of elevator group control systems for vertical transportation: A look at recent literature. *IEEE Control Syst. Mag.* **2015**, *35*, 38–55.
3. Strakosch, G.R.; Caporale, R.S. *The Vertical Transportation Handbook*, 4th ed.; Wiley: Hoboken, NJ, USA, 2010.
4. Siikonen, M.L. Planning and Control Models for Elevators in High-Rise Buildings. Ph.D. Thesis, Helsinki University of Technology, Helsinki, Finland, 1997.
5. Fujino, A.; Tobita, T.; Segawa, K.; Yoneda, K.; Togawa, A. An elevator group control system with floor-attribute control method and system optimization using genetic algorithms. *IEEE Trans. Ind. Electron.* **1997**, *44*, 546–552. [\[CrossRef\]](#)
6. Cortés, P.; Larrañeta, J.; Onieva, L. Genetic algorithm for controllers in elevator groups: Analysis and simulation during lunchpeak traffic. *Appl. Soft Comput.* **2004**, *4*, 159–174. [\[CrossRef\]](#)
7. Jamaludin, J.; Rahim, N.A.; Hew, W.P. Development of a self-tuning fuzzy logic controller for intelligent control of elevator systems. *Eng. Appl. Artif. Intell.* **2009**, *22*, 1167–1178. [\[CrossRef\]](#)
8. Liu, J.; Bai, Z.L.; Gu, M.H.; Zhang, X.; Zhang, R. The research of multi-car elevator control method based on PSO-GA. *Appl. Mech. Mater.* **2014**, *556*, 2418–2421. [\[CrossRef\]](#)
9. Wei, Q.; Wang, L.; Liu, Y.; Polycarpou, M.M. Optimal elevator group control via deep asynchronous actor–critic learning. *IEEE Trans. Neural Netw. Learn. Syst.* **2020**, *31*, 5245–5256. [\[CrossRef\]](#) [\[PubMed\]](#)
10. Li, H.Y.; Chu, E.T.H. SmartRide: Intelligent reservation and scheduling for elevators. *J. Ambient Intell. Smart Environ.* **2025**, *17*, 75–100. [\[CrossRef\]](#)
11. Wang, S.; Gong, X.; Song, M.; Fei, C.Y.; Quaadgras, S.; Peng, J.; Zou, P.; Chen, J.; Zhang, W.; Jiao, R.J. Smart dispatching and optimal elevator group control through real-time occupancy-aware deep learning of usage patterns. *Adv. Eng. Inform.* **2021**, *48*, 101286. [\[CrossRef\]](#)
12. Maleki, E.F.; Bhatta, D.; Mashayekhy, L. A game-theoretic approach to energy-efficient elevator scheduling in smart buildings. *IEEE Trans. Syst. Man Cybern. Syst.* **2023**, *53*, 3944–3955. [\[CrossRef\]](#)
13. Wu, Y.; Yang, J. Directional optimization of elevator scheduling algorithms in complex traffic patterns. *Appl. Soft Comput.* **2024**, *158*, 111567. [\[CrossRef\]](#)

14. Chen, W.; Zheng, B.; Liu, J.; Li, L.; Ren, X. A real-time matrix iterative optimization algorithm of booking elevator group and numerical simulation formed by multi-sensor combination. *Electronics* **2021**, *10*, 3179. [[CrossRef](#)]
15. Zhang, J.; Tsiligkaridis, A.; Taguchi, H.; Raghunathan, A.; Nikovski, D. Transformer networks for predictive group elevator control. *arXiv* **2022**, arXiv:2208.08948. [[CrossRef](#)]
16. Wan, J.; Lee, K.; Shin, H. Traffic pattern-aware elevator dispatching via deep reinforcement learning. *Adv. Eng. Inform.* **2024**, *61*, 102497. [[CrossRef](#)]
17. Talebi, M. Application of fuzzy-enhanced reinforcement learning in AI for elevator group control. *Elev. World* **2025**, *73*, 120.
18. Rashed, A.N.Z.; Yarrarapu, M.; Prabu, R.T.; Raj Antony, G.S.; Edeswaran, L.; Kumar, E.S.; Aswitha, K.; Snehith, N.; Ahammad, S.H. Connected smart elevator systems for smart power and time saving. *Sci. Rep.* **2024**, *14*, 19330. [[CrossRef](#)] [[PubMed](#)]
19. Sahin, Y.G.; Uzunbayir, S.; Akcay, B.; Yildiz, E. Real-time monitoring of elevators to reduce redundant stops and energy wastage. In Proceedings of the 2013 International Conference on Systems, Control and Informatics, Venice, Italy, 28–30 September 2013.
20. Wang, S.; Gong, X.; Zhang, W.; Chen, J.; Jiao, R.J. Smart elevator dispatching and optimization through deep learning of usage patterns. In Proceedings of the IISE Annual Conference, Orlando, FL, USA, 18–21 May 2019; pp. 1498–1503.
21. Pan, J.; Shao, C.; Dai, Y.; Wei, Y.; Chen, W.; Lin, Z. Research on fault prediction method of elevator door system based on transfer learning. *Sensors* **2024**, *24*, 2135. [[CrossRef](#)] [[PubMed](#)]
22. Shi, M.; Choi, Y. Comparison of the elevator traffic flow prediction between the neural networks of CNN and LSTM. *Intell. Control Syst. Eng.* **2024**, *2*, 1871.
23. Bolat, B.; Cortés, P.; Yalçın, E.; Alişverişiçi, M. Optimal car dispatching for elevator groups using genetic algorithms. *Intell. Autom. Soft Comput.* **2010**, *16*, 89–99. [[CrossRef](#)]
24. Hautamäki, T. Multiobjective Optimization Model for Elevator Call Allocation. Master's Thesis, Aalto University, Espoo, Finland, 2021.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction
prohibited without permission.